KELAS QUERY

SQL untuk Pemula

Rantai pasok logistik kesehatan

DDL SELECT JOIN SUBQUERY CTE

Schema bootcamp · PostgreSQL

Hasil belajar dan alur kelas

Bagian	Peserta dapat…
01 Struktur data	Membuat tabel dan memilih tipe data
02 Integritas & relasi	Menjelaskan PK, FK, dan constraint
03 SELECT dasar	Memilih, memfilter, dan mengurutkan baris
04 Analisis	Menghitung agregat dan menggabungkan tabel
05 Query bertahap	Memakai subquery dan CTE

Saran pengajaran: dua pertemuan @ 2 jam, termasuk demo dan latihan.

Database, schema, tabel, dan baris

Istilah	Arti	Contoh kelas
Database	Wadah utama objek PostgreSQL	Database tempat bootcamp dibuat
Schema	Namespace di dalam database	bootcamp
Tabel	Kumpulan baris dengan struktur sama	bootcamp.stocks
Kolom	Atribut dengan tipe data	entity_id, qty_on_hand
Baris / grain	Satu catatan sesuai detail tabel	Satu catatan stok

bootcamp adalah schema. SET search_path memilih urutan schema untuk pencarian nama tabel.

Kelompok perintah SQL

Kelompok	Tujuan	Contoh
DDL	Mendefinisikan struktur	CREATE, ALTER, DROP, TRUNCATE
DML	Menambah atau mengubah isi	INSERT, UPDATE, DELETE
Query / sering disebut DQL	Membaca data	SELECT
DCL	Mengatur akses	GRANT, REVOKE
TCL	Mengatur transaksi	COMMIT, ROLLBACK

Pengelompokan istilah bisa berbeda antarreferensi. Fokus kelas: DDL dan SELECT.

Peta tabel master dan referensi

Area	Tabel
Wilayah & fasilitas	locations, entity_types, entities
Pengguna & kegiatan	users, activities, entity_activities
Material	material_types, material_units, materials, batches
Referensi pesanan	order_types, order_statuses, delivery_types
Referensi mutasi	transaction_types, transaction_reasons

Contoh latihan memakai nama dan kolom dari skema bootcamp, dengan nilai data mini yang fiktif.

Grain tabel operasional

Tabel	Makna satu baris	Hal yang diperhatikan
stocks	Satu catatan stok per kombinasi dimensi	entity, activity, material, batch
orders	Satu dokumen permintaan / pengiriman	customer dan vendor adalah entities
order_items	Satu baris material dalam pesanan	PK id, pasangan order-material bisa berulang
transactions	Satu catatan mutasi stok	opening + change = closing menurut komentar

Snapshot qty_on_hand dan akumulasi change_qty menjawab pertanyaan yang berbeda.

Relasi pada skema bootcamp

Induk	Anak / kolom referensi	Makna
entities.id	orders.customer_id	Fasilitas yang meminta
entities.id	orders.vendor_id	Fasilitas yang mengirim
orders.id	order_items.order_id	Pesanan memiliki 0..N baris item
materials.id	stocks.material_id	Material memiliki 0..N catatan stok
stocks.id	transactions.stock_id	Stok memiliki 0..N catatan mutasi
entities dan activities	Melalui entity_activities	Relasi N:M

Setiap FK menjaga referensi masing-masing. FK tidak menjamin konsistensi seluruh kombinasi bisnis.

Tipe data dari DDL bootcamp

Tipe PostgreSQL	Contoh kolom	Kegunaan
SMALLINT / INT	status, entity_id	Bilangan bulat dengan rentang berbeda
BIGINT	qty_on_hand, change_qty	Bilangan bulat untuk kuantitas / ID
TEXT	name, no_document	Teks dengan panjang bervariasi
NUMERIC(p,s)	lat NUMERIC(10,6)	Desimal dengan presisi tetap
DATE / TIMESTAMP	expired_date, created_at	Tanggal / tanggal dan waktu
UUID	transactions.uuid	Identifier berformat UUID

TIMESTAMP pada DDL ini berarti timestamp tanpa zona waktu. UUID NOT NULL belum berarti UNIQUE.

Primary key dan foreign key

Aspek	Primary key	Foreign key
Fungsi	Identitas unik baris	Referensi ke baris induk
Duplikasi	Nilai PK tidak boleh berulang	FK boleh berulang
NULL	Tidak boleh	Boleh jika tidak ada NOT NULL
Contoh	orders.id	orders.customer_id
Composite key	PK dapat memakai beberapa kolom	FK dapat merujuk key gabungan

order_items.id adalah PK. DDL ini tidak memiliki PK gabungan (order_id, material_id).

Constraint dan nilai default

Aturan	Contoh asli	Makna
PRIMARY KEY	transactions.id	Identitas unik dan tidak NULL
FOREIGN KEY	stocks.entity_id REFERENCES entities(id)	Nilai non-NULL merujuk induk
NOT NULL	materials.name	Nilai wajib tersedia
UNIQUE	entities.code	Kode unik
UNIQUE gabungan	entity_activities(entity_id, activity_id)	Pasangan fasilitas-kegiatan unik
DEFAULT	stocks.qty_on_hand DEFAULT 0	Nilai awal jika kolom tidak diberikan

CHECK belum ada pada DDL bootcamp. DEFAULT memberi nilai awal tanpa membatasi seluruh nilai.

DDL: schema dan search_path

```
CREATE SCHEMA bootcamp;
SET search_path TO bootcamp;

-- Nama dengan schema eksplisit
SELECT id, name
FROM bootcamp.entities
ORDER BY id;
```

CREATE SCHEMA membuat namespace.

search_path berlaku pada koneksi / sesi.

Nama bootcamp.entities tetap jelas walau search_path berbeda.

DDL asli juga memiliki DROP SCHEMA ... CASCADE. Gunakan hanya saat sengaja mereset lingkungan latihan.

DDL: lokasi dengan self-reference

```
CREATE TABLE locations (
  id INT PRIMARY KEY,
  name TEXT NOT NULL,
  level SMALLINT NOT NULL,
  parent_id INT REFERENCES locations(id),
  code TEXT NOT NULL
);
```

parent_id merujuk baris lain di tabel locations.

Provinsi bisa memiliki parent_id NULL.

Komentar DDL: level 0 provinsi, 1 kab/kota, 2 kecamatan.

FK tidak membatasi level menjadi 0–2 dan tidak mencegah siklus hierarki.

DDL: tabel referensi jenis fasilitas

```
CREATE TABLE entity_types (
  id SMALLINT PRIMARY KEY,
  name TEXT NOT NULL,
  code TEXT NOT NULL
);
```

id	name
1	Dinkes Prov
2	Dinkes Kab
3	Puskesmas

Tabel kanan adalah contoh isi mini.

Arti ID pada data generator asli harus dibaca dari entity_types.

DDL menentukan struktur. Nilai ID referensi bukan kamus status universal.

DDL: membaca tabel entities yang panjang

```
-- Cuplikan DDL asli, kolom lain disingkat
id INT PRIMARY KEY,
code TEXT NOT NULL UNIQUE,
entity_type_id SMALLINT NOT NULL
  REFERENCES entity_types(id),
parent_entity_id INT REFERENCES entities(id),
province_id INT REFERENCES locations(id),
status SMALLINT NOT NULL DEFAULT 1
```

entity_type_id wajib memiliki pasangan referensi.

parent_entity_id membentuk hierarki fasilitas.

DEFAULT 1 tidak melarang nilai status lain.

Cuplikan ini menjelaskan bagian tabel, bukan statement CREATE TABLE yang lengkap.

DDL: relasi N:M melalui entity_activities

```
-- Cuplikan definisi entity_activities
id INT PRIMARY KEY,
entity_id INT NOT NULL REFERENCES entities(id),
activity_id SMALLINT NOT NULL
  REFERENCES activities(id),
UNIQUE (entity_id, activity_id)
```

Satu fasilitas dapat mengikuti banyak kegiatan.

Satu kegiatan dapat diikuti banyak fasilitas.

Pasangan yang sama tidak boleh masuk dua kali.

id tetap menjadi PK. UNIQUE gabungan menjaga aturan pasangan yang berbeda dari identitas PK.

Aturan bisnis dan constraint aktual

```
-- Audit, tanpa mengubah tabel
SELECT id
FROM transactions
WHERE closing_qty <> opening_qty + change_qty;

-- Usulan tambahan, bukan DDL asli
ALTER TABLE transactions
ADD CONSTRAINT chk_qty_balance
CHECK (closing_qty = opening_qty + change_qty);
```

Komentar “selalu =” belum membuat database memeriksa persamaan.

Audit mencari pelanggaran.

CHECK adalah opsi perubahan desain setelah data dan aturan disepakati.

Kolom NOT NULL membuat pemeriksaan persamaan ini tidak bergantung pada perilaku CHECK terhadap NULL.

ALTER, DROP, TRUNCATE, dan DELETE

Perintah	Efek	Contoh
ALTER TABLE	Mengubah struktur / aturan	ALTER TABLE ... ADD CONSTRAINT ...
DROP TABLE	Menghapus tabel beserta isinya	DROP TABLE tabel_demo;
TRUNCATE TABLE	Mengosongkan seluruh tabel	TRUNCATE TABLE tabel_demo;
DELETE FROM	Menghapus baris sesuai filter	DELETE FROM tabel_demo WHERE id = 1;

Contoh sintaks PostgreSQL. TRUNCATE tidak memiliki WHERE dan memperhatikan dependensi FK.

Referential action pada foreign key

Aksi	Perilaku saat induk dihapus
NO ACTION (default)	Menolak jika referensi anak masih melanggar FK saat pemeriksaan
RESTRICT	Menolak penghapusan induk yang masih dirujuk
CASCADE	Ikut menghapus baris anak yang merujuk induk
SET NULL	Mengubah FK anak menjadi NULL jika kolom mengizinkannya

DDL bootcamp tidak mendeklarasikan ON DELETE CASCADE. DROP SCHEMA CASCADE adalah operasi yang berbeda.

INSERT: mengisi tabel referensi

```
INSERT INTO transaction_types
  (id, name, direction)
VALUES
  (1, 'Penerimaan', 1),
  (2, 'Pengeluaran', -1);
```

id	name	direction
1	Penerimaan	1
2	Pengeluaran	-1

Isi ini adalah kamus data mini.

Isi induk sebelum anak.

transaction_reasons memerlukan transaction_type_id yang sudah tersedia.

Contoh INSERT untuk tabel referensi kosong. Pada data generator yang sudah terisi, periksa ID terlebih dahulu.

Data mini: fasilitas dan material

entities (kolom terpilih)

id	name	type_id*
1	Dinkes Prov	1
2	Dinkes Kab	2
3	PKM Melati	3
4	PKM Mawar	3

materials (kolom terpilih)

id	name	unit
1000	Vaksin A	dosis
1001	Syringe	pcs
1002	Safety box	box

***type_id menyingkat entity_type_id. Label unit berasal dari material_units. Nilai hanya ilustrasi.**

Data mini: snapshot stok

id	entity_id	material_id	batch_id	qty_on_hand
100	2	1000	5000	800
101	3	1000	5000	120
102	3	1000	5001	80
103	4	1000	5001	0
104	3	1001	5002	500

Semua baris stok contoh memakai activity_id = 1. Saldo Vaksin A di PKM Melati = 200 dosis.

Data mini: pesanan dan status

id	customer	vendor	status_id	created_at	deleted_at
201	3	2	3	2026-02-01	NULL
202	4	2	2	2026-02-10	NULL
203	3	2	1	2026-02-15	NULL
204	3	2	3	2026-01-20	NULL
205	4	2	1	2026-02-18	2026-02-20

Kamus status data mini: 1 Draft, 2 Dikirim, 3 Diterima. Semua contoh laporan mengabaikan soft delete.

Data mini: baris material dalam pesanan

id	order_id	material_id	requested	approved	sent
301	201	1000	200	180	180
302	201	1001	100	100	100
303	202	1000	150	150	150
304	203	1000	100	NULL	NULL
305	204	1000	50	50	50
306	205	1000	70	NULL	NULL

requested, approved, dan sent di tabel ini menyingkat qty_requested, qty_approved, qty_sent.

SELECT: memilih kolom

```
SELECT id, name
FROM entities
ORDER BY id;
```

id	name
1	Dinkes Prov
2	Dinkes Kab
3	PKM Melati
4	PKM Mawar

SELECT memilih kolom.
FROM memilih sumber.
ORDER BY menetapkan urutan.

SELECT * mengambil semua kolom. Pilih kolom yang diperlukan agar maksud query jelas.

Alias dan kolom hasil perhitungan

```
SELECT id,
       opening_qty + change_qty AS expected
FROM transactions
WHERE id IN (9001, 9002)
ORDER BY id;
```

id	expected
9001	180
9002	120

AS memberi nama hasil ekspresi.

Alias tidak menambah kolom permanen ke tabel.

Contoh mutasi: 0 + 180 dan 180 − 60.

Teks memakai tanda kutip tunggal. Alias dan nama kolom harus sesuai query block.

WHERE: memfilter catatan stok

```
SELECT id, qty_on_hand
FROM stocks
WHERE entity_id = 3
  AND material_id = 1000
  AND activity_id = 1
  AND qty_on_hand > 0
ORDER BY id;
```

id	qty_on_hand
101	120
102	80

Hanya baris dengan kondisi TRUE yang lolos.

Operator umum:
= <> > >= < <=

Filter dimensi menentukan arti jumlah yang akan dihitung.

AND, OR, NOT, dan tanda kurung

```
SELECT id, customer_id AS cust_id
FROM orders
WHERE deleted_at IS NULL
  AND (customer_id = 3 OR customer_id = 4)
  AND NOT (order_status_id = 1)
ORDER BY id;
```

id	cust_id
201	3
202	4
204	3

AND: semua syarat benar.
OR: minimal satu benar.
NOT: membalik kondisi.

Prioritas: NOT, AND, OR.

ID status 1 berarti Draft hanya pada data mini. Pada data asli, baca tabel order_statuses.

IN, BETWEEN, LIKE, dan ILIKE

```
-- Pilihan fasilitas
WHERE entity_id IN (3, 4)

-- Batas bawah dan atas ikut masuk
WHERE qty_on_hand BETWEEN 1 AND 200

-- Pencarian nama
WHERE name LIKE 'PKM%'
WHERE name ILIKE 'pkm%'
```

IN meringkas beberapa OR.

BETWEEN bersifat inklusif.

% untuk nol atau lebih karakter.
_ untuk satu karakter.

ILIKE adalah pencarian tanpa membedakan huruf pada PostgreSQL.

Potongan kondisi harus dilengkapi SELECT dan FROM sesuai tabelnya.

NULL dan COALESCE

```
SELECT id,
       COALESCE(qty_approved, 0) AS approved
FROM order_items
WHERE qty_approved IS NULL
ORDER BY id;
```

id	approved
304	0
306	0

IS NULL memeriksa nilai yang belum tersedia.

= NULL tidak memberi TRUE.

COALESCE memberi pengganti saat menampilkan atau menghitung.

NULL pada qty_approved berarti belum tersedia. Mengubah tampilannya menjadi 0 tidak mengubah makna asal.

Filter tanggal dengan rentang

```
SELECT id, no_document
FROM orders
WHERE created_at >= TIMESTAMP '2026-02-01'
  AND created_at <  TIMESTAMP '2026-03-01'
  AND deleted_at IS NULL
ORDER BY created_at, id;
```

id	no_document
201	ORD-201
202	ORD-202
203	ORD-203

Rentang ini mengambil seluruh Februari.

Batas < 1 Maret mencakup jam terakhir di Februari tanpa menebak presisi timestamp.

Kolom TIMESTAMP pada skema ini tidak menyimpan zona waktu. Samakan konvensi waktu sumber dan laporan.

DISTINCT: kombinasi nilai unik

```
SELECT DISTINCT entity_id
FROM stocks
WHERE material_id = 1000
  AND activity_id = 1
ORDER BY entity_id;
```

entity_id
2
3
4

Dua batch di PKM Melati tetap menghasilkan satu entity_id.

Jika batch_id ikut SELECT, DISTINCT mengevaluasi kombinasi kedua kolom.

DISTINCT tidak memperbaiki kunci JOIN atau grain yang keliru.

ORDER BY dan LIMIT

```
SELECT id, qty_on_hand
FROM stocks
WHERE material_id = 1000
  AND activity_id = 1
ORDER BY qty_on_hand DESC, id ASC
LIMIT 2;
```

id	qty_on_hand
100	800
101	120

DESC: besar ke kecil.
ASC: kecil ke besar.

id menjadi pemecah seri ketika qty_on_hand sama.

Tanpa ORDER BY, urutan hasil tidak dijamin. LIMIT membatasi jumlah baris keluaran.

Latihan 1: SELECT dan filter

1. Tampilkan fasilitas yang namanya diawali PKM, urutkan nama A–Z.
2. Cari stok material 1000 dan activity 1 dengan qty 1 sampai 200.
3. Cari order_items yang qty_approved-nya belum tersedia.
4. Ambil dua pesanan nondeleted terbaru berdasarkan created_at dan id.

Waktu latihan: 10 menit. Gunakan ID material dan activity contoh, atau sesuaikan dengan data kelas.

Agregasi pada satu material dan kegiatan

```
SELECT COUNT(*) AS rows,
       SUM(qty_on_hand) AS total,
       MIN(qty_on_hand) AS min_qty,
       MAX(qty_on_hand) AS max_qty,
       AVG(qty_on_hand) AS avg_qty
FROM stocks
WHERE material_id = 1000
  AND activity_id = 1;
```

Hasil data mini:
rows = 4
total = 1.000 dosis
min_qty = 0
max_qty = 800
avg_qty = 250 per baris stok

Rata-rata per baris stok berbeda dari rata-rata per fasilitas. Satu fasilitas dapat punya beberapa batch.

COUNT(*) dan COUNT(kolom)

```
SELECT COUNT(*) AS rows,
       COUNT(qty_approved) AS filled,
       COUNT(DISTINCT material_id) AS mats
FROM order_items;
```

rows	filled	mats
6	4	2

COUNT(*) menghitung semua baris.

COUNT(qty_approved) mengabaikan NULL.

COUNT(DISTINCT material_id) menghitung material unik non-NULL.

Contoh ini sengaja menghitung semua item, termasuk item dari pesanan soft-deleted.

GROUP BY: saldo per fasilitas

```
SELECT entity_id,
       SUM(qty_on_hand) AS total
FROM stocks
WHERE material_id = 1000
  AND activity_id = 1
GROUP BY entity_id
ORDER BY entity_id;
```

entity_id	total
2	800
3	200
4	0

Empat catatan stok menjadi tiga kelompok fasilitas.

Dua batch PKM Melati digabung menjadi 200 dosis.

Aturan aman: kolom SELECT yang tidak diagregasi ikut GROUP BY.

HAVING: memfilter kelompok fasilitas

```
SELECT entity_id,
       SUM(qty_on_hand) AS total
FROM stocks
WHERE material_id = 1000
  AND activity_id = 1
GROUP BY entity_id
HAVING SUM(qty_on_hand) >= 200
ORDER BY entity_id;
```

entity_id	total
2	800
3	200

GROUP BY membentuk kelompok.

SUM menghitung saldo setiap kelompok.

HAVING memilih total minimal 200 dosis.

WHERE SUM(qty_on_hand) >= 200 tidak valid untuk agregat pada query block yang sama.

WHERE dan HAVING pada query yang sama

```
SELECT entity_id,
       SUM(qty_on_hand) AS total
FROM stocks
WHERE material_id = 1000
  AND activity_id = 1
  AND qty_on_hand > 0
GROUP BY entity_id
HAVING SUM(qty_on_hand) >= 200
ORDER BY total DESC, entity_id;
```

entity_id	total
2	800
3	200

WHERE menentukan catatan stok yang ikut dihitung.

HAVING memfilter hasil kelompok.

Fasilitas dengan nol stok tidak ada dalam kelompok karena filter WHERE.

WHERE bekerja pada baris. HAVING bekerja pada hasil kelompok.

Urutan tulis dan urutan logis SELECT

Urutan tulis	Urutan logis untuk query inti kelas
SELECT	1. FROM dan JOIN
FROM / JOIN	2. WHERE
WHERE	3. GROUP BY dan agregasi
GROUP BY	4. HAVING
HAVING	5. SELECT, lalu DISTINCT bila ada
ORDER BY	6. ORDER BY
LIMIT	7. LIMIT

Ini model untuk memahami hasil query. Optimizer dapat memilih urutan kerja fisik yang berbeda.

CASE WHEN: kategori saldo stok

```
SELECT id,
       CASE
         WHEN qty_on_hand = 0 THEN 'Kosong'
         WHEN qty_on_hand < 100 THEN 'Rendah'
         ELSE 'Tersedia'
       END AS label
FROM stocks
WHERE material_id = 1000
  AND activity_id = 1
ORDER BY id;
```

id	label
100	Tersedia
101	Tersedia
102	Rendah
103	Kosong

Batas 100 hanya aturan latihan pada tingkat baris stok. Ini bukan definisi kecukupan stok produksi.

Latihan 2: GROUP BY dan HAVING

1. Hitung pesanan nondeleted per order_status_id.

2. Hitung pesanan nondeleted per customer_id, lalu tampilkan pelanggan dengan lebih dari satu pesanan.

3. Hitung saldo material 1000 pada activity 1 per entity_id.

4. Mengapa WHERE SUM(qty_on_hand) > 200 tidak valid untuk agregasi langsung?

Waktu latihan: 10–15 menit. Nyatakan grain hasil dan filter data yang ikut dihitung.

JOIN: membaca tabel yang berelasi

Bentuk JOIN	Baris yang dipertahankan	Contoh
INNER JOIN	Hanya pasangan yang cocok	Stok dengan nama material
LEFT JOIN	Semua baris kiri	Semua fasilitas, termasuk tanpa order
RIGHT JOIN	Semua baris kanan	Dapat ditulis ulang dengan LEFT JOIN
CROSS JOIN	Semua kombinasi	Fasilitas × kegiatan
SELF JOIN	Tabel yang sama memakai alias berbeda	Fasilitas dan induknya

Self join adalah pola memakai tabel yang sama, bukan keyword JOIN tersendiri.

INNER JOIN: pesanan dan fasilitas peminta

```
SELECT o.id, e.name
FROM orders AS o
JOIN entities AS e ON e.id = o.customer_id
WHERE o.deleted_at IS NULL
ORDER BY o.id;
```

id	name
201	PKM Melati
202	PKM Mawar
203	PKM Melati
204	PKM Melati

customer_id pada orders merujuk entities.

Tidak ada tabel customers pada skema ini.

Alias membantu membedakan kolom bernama id dan name pada banyak tabel.

Dua peran entities pada satu pesanan

```
SELECT o.id,
       c.name AS customer,
       v.name AS vendor
FROM orders AS o
JOIN entities AS c ON c.id = o.customer_id
JOIN entities AS v ON v.id = o.vendor_id
WHERE o.id = 201;
```

id	customer	vendor
201	PKM Melati	Dinkes Kab

Alias c mewakili peminta.

Alias v mewakili pengirim.

Keduanya membaca tabel entities.

Nama vendor_id menyatakan peran bisnis. FK tidak memeriksa is_vendor = 1.

LEFT JOIN: fasilitas tanpa pesanan

```
SELECT e.id, e.name
FROM entities AS e
LEFT JOIN orders AS o
  ON o.customer_id = e.id
 AND o.deleted_at IS NULL
WHERE o.id IS NULL
ORDER BY e.id;
```

id	name
1	Dinkes Prov
2	Dinkes Kab

Fasilitas tetap muncul meski tidak memiliki order sebagai customer.

Dinkes Kab punya peran vendor, tetapi tidak punya order sebagai customer dalam data mini.

“Tanpa pesanan” pada query ini berarti tanpa pesanan nondeleted sebagai peminta.

Filter ON dan WHERE pada LEFT JOIN

```
SELECT e.id, o.id AS order_id
FROM entities AS e
LEFT JOIN orders AS o
  ON o.customer_id = e.id
 AND o.deleted_at IS NULL
 AND o.received_at IS NOT NULL
WHERE e.id IN (3, 4)
ORDER BY e.id, o.id;
```

id	order_id
3	201
3	204
4	NULL

Filter received_at di ON membatasi pasangan pesanan.

Memindah filter received_at IS NOT NULL ke WHERE akan membuang PKM Mawar.

Filter tabel kiri di WHERE berbeda dari filter pasangan tabel kanan di ON.

JOIN beberapa tabel: jumlah terkirim

```
SELECT e.name, SUM(i.qty_sent) AS sent
FROM orders AS o
JOIN entities AS e ON e.id = o.customer_id
JOIN order_items AS i ON i.order_id = o.id
WHERE o.deleted_at IS NULL
  AND o.activity_id = 1
  AND i.material_id = 1000
GROUP BY e.id, e.name
ORDER BY sent DESC NULLS LAST, e.id;
```

name	sent
PKM Melati	230
PKM Mawar	150

qty_sent berasal dari item pesanan.

SUM mengabaikan NULL.

Laporan ini memakai semua status nondeleted dan material yang sama.

Jumlah terkirim bukan jumlah diterima. Skema tidak memiliki qty_received pada order_items.

Risiko JOIN: satu order menjadi banyak baris

```
SELECT COUNT(*) AS rows,
       COUNT(DISTINCT o.id) AS orders
FROM orders AS o
JOIN order_items AS i ON i.order_id = o.id
WHERE o.deleted_at IS NULL;
```

rows	orders
5	4

Order 201 memiliki dua item.

Empat order nondeleted menghasilkan lima baris hasil JOIN.

COUNT(*) setelah JOIN menghitung baris JOIN. Hindari SUM(total_order_items) setelah JOIN ke item.

LEFT JOIN dan COUNT untuk hasil nol

```
SELECT e.name, COUNT(o.id) AS total
FROM entities AS e
LEFT JOIN orders AS o
  ON o.customer_id = e.id
 AND o.deleted_at IS NULL
GROUP BY e.id, e.name
ORDER BY e.id;
```

name	total
Dinkes Prov	0
Dinkes Kab	0
PKM Melati	3
PKM Mawar	1

COUNT(o.id) mengabaikan NULL untuk fasilitas tanpa pasangan.

COUNT(*) menghitung baris placeholder hasil outer join.

Satu baris hasil LEFT JOIN tidak selalu berarti satu order.

UNION dan UNION ALL

```
SELECT name AS label FROM entities
WHERE id IN (3, 4)
UNION ALL
SELECT name AS label FROM entities
WHERE id = 3
ORDER BY label;
```

label
PKM Mawar
PKM Melati
PKM Melati

UNION ALL menumpuk baris dan mempertahankan duplikasi.

UNION menghapus duplikasi.

Jumlah kolom harus sama dengan tipe yang kompatibel.

JOIN mencocokkan baris dari sumber berbeda. UNION menumpuk baris hasil query.

Subquery: query di dalam query

Bentuk	Lokasi umum	Tujuan contoh
Scalar	SELECT atau pembanding WHERE	Stok di atas rata-rata
Daftar / set	IN / NOT IN	Fasilitas yang memiliki stok
Derived table	FROM	Filter saldo hasil agregasi
Correlated	Merujuk kolom query luar	Material tanpa stok

Jumlah baris dan kolom hasil subquery harus sesuai dengan tempat pemakaiannya.

Scalar subquery: stok di atas rata-rata

```
SELECT id, qty_on_hand
FROM stocks
WHERE material_id = 1000 AND activity_id = 1
  AND qty_on_hand > (
    SELECT AVG(qty_on_hand)
    FROM stocks
    WHERE material_id = 1000
      AND activity_id = 1
  )
ORDER BY id;
```

id	qty_on_hand
100	800

Rata-rata empat catatan stok = 250 dosis.

Subquery menghasilkan satu nilai agregat.

Query luar mencari baris di atas nilai itu.

Bandingkan dimensi dan grain yang sama antara query luar dan subquery.

Subquery IN: fasilitas yang punya stok

```
SELECT id, name
FROM entities
WHERE id IN (
  SELECT entity_id FROM stocks
  WHERE material_id = 1000
    AND activity_id = 1
    AND qty_on_hand > 0
)
ORDER BY id;
```

id	name
2	Dinkes Kab
3	PKM Melati

IN memeriksa keanggotaan ID.

Beberapa batch dalam subquery tidak menggandakan baris fasilitas.

IN cocok untuk pertanyaan apakah sebuah nilai berada dalam kumpulan.

EXISTS dan correlated subquery

```
SELECT m.id, m.name
FROM materials AS m
WHERE NOT EXISTS (
  SELECT 1 FROM stocks AS s
  WHERE s.material_id = m.id
)
ORDER BY m.id;
```

id	name
1002	Safety box

Subquery merujuk m.id dari query luar.

NOT EXISTS mencari material tanpa satu pun catatan stok.

Tidak punya catatan stok berbeda dari punya catatan dengan qty_on_hand = 0.

NULL pada NOT IN

```
-- Ilustrasi ekspresi NULL
SELECT 1002 NOT IN (1000, 1001, NULL)
  AS result;
```

result
NULL (UNKNOWN)

Jika daftar berisi NULL, NOT IN dapat menghasilkan UNKNOWN.

WHERE hanya meloloskan TRUE.

NOT EXISTS sering lebih jelas untuk mencari baris tanpa pasangan.

stocks.material_id NOT NULL. Namun stocks.batch_id dan transactions.order_id dapat berisi NULL.

Derived table: saldo hasil agregasi

```
SELECT x.entity_id, x.total
FROM (
  SELECT entity_id, SUM(qty_on_hand) AS total
  FROM stocks
  WHERE material_id = 1000 AND activity_id = 1
  GROUP BY entity_id
) AS x
WHERE x.total >= 200
ORDER BY x.entity_id;
```

entity_id	total
2	800
3	200

Bagian dalam menghasilkan total per fasilitas.

Query luar memakai total sebagai kolom dari sumber x.

WHERE boleh memfilter hasil agregasi yang sudah menjadi keluaran query block lain.

CTE: memberi nama pada tahap query

```
WITH stock_summary AS (
  SELECT entity_id, SUM(qty_on_hand) AS total
  FROM stocks
  WHERE material_id = 1000 AND activity_id = 1
  GROUP BY entity_id
)
SELECT entity_id, total
FROM stock_summary
WHERE total >= 200
ORDER BY entity_id;
```

entity_id	total
2	800
3	200

WITH memberi nama pada hasil query.

CTE berlaku hanya dalam satu statement.

Hasilnya sama dengan derived table sebelumnya.

CTE membantu keterbacaan. CTE bukan tabel permanen dan tidak otomatis lebih cepat.

Beberapa CTE untuk laporan bertahap

```
WITH eligible AS (
  SELECT id, customer_id FROM orders
  WHERE deleted_at IS NULL AND activity_id = 1
), sent_qty AS (
  SELECT o.customer_id, SUM(i.qty_sent) AS sent
  FROM eligible AS o
  JOIN order_items AS i ON i.order_id = o.id
  WHERE i.material_id = 1000
  GROUP BY o.customer_id
)
SELECT customer_id, sent FROM sent_qty
WHERE sent >= 200
ORDER BY sent DESC, customer_id;
```

customer_id	sent
3	230

Tahap 1 memilih order yang relevan.

Tahap 2 menghitung qty terkirim.

Query utama memilih total minimal 200 dosis.

Antar-CTE dipisahkan koma. Akhiri statement setelah query utama.

JOIN, subquery, atau CTE?

Pilihan	Paling membantu ketika…	Hal yang diperiksa
JOIN	Perlu kolom dari beberapa sumber	Kardinalitas dan duplikasi
IN / EXISTS	Perlu memeriksa keanggotaan / keberadaan	NULL dan korelasi
Derived table	Hasil query langsung menjadi sumber	Alias dan grain hasil
CTE	Query memiliki beberapa tahap yang bernama	Scope dan rencana eksekusi

Pilih bentuk yang jelas dan benar. Bandingkan performa menggunakan EXPLAIN pada database target.

Kesalahan umum pada skema ini

Gejala	Kemungkinan penyebab	Pemeriksaan
Unknown column	Memakai kolom dari contoh lama	Gunakan qty_on_hand, qty_sent, created_at
Hasil nol terlalu besar	COUNT(*) setelah LEFT JOIN	Gunakan COUNT(o.id)
Jumlah membesar	JOIN stok ke mutasi atau order ke item	Grain dan kardinalitas
Status keliru	Menebak arti ID numerik	JOIN ke tabel referensi
Jumlah campur unit	Menjumlah dosis dan pcs	Filter material / unit yang sebanding
Pesanan terhapus ikut laporan	Soft delete belum difilter	deleted_at IS NULL

Satu hasil numerik yang valid secara SQL belum tentu menjawab pertanyaan bisnis yang benar.

Latihan akhir: laporan stok dan distribusi

1. Tampilkan semua fasilitas beserta jumlah order nondeleted sebagai customer, termasuk nol.

2. Hitung qty_sent material 1000 pada activity 1 per customer dari order nondeleted. Tampilkan total minimal 200.

3. Cari material yang tidak memiliki catatan stocks dengan NOT EXISTS.

4. Tulis ulang soal 2 memakai CTE dan bandingkan hasilnya.

Waktu latihan: 20 menit. Pada data asli, angka hasil mengikuti data generator yang digunakan.

Pembahasan latihan 1: filter dan NULL

```
-- 1. Fasilitas PKM
SELECT name FROM entities
WHERE name LIKE 'PKM%' ORDER BY name;

-- 2. Catatan stok dalam rentang
SELECT id, qty_on_hand FROM stocks
WHERE material_id = 1000 AND activity_id = 1
  AND qty_on_hand BETWEEN 1 AND 200
ORDER BY id;

-- 3. Qty persetujuan belum tersedia
SELECT id FROM order_items
WHERE qty_approved IS NULL ORDER BY id;
```

Hasil data mini:

Soal 1: PKM Mawar, PKM Melati

Soal 2: id 101 = 120, id 102 = 80

Soal 3: id 304 dan 306

Query item di soal 3 tidak memfilter order soft-deleted karena fokusnya mencari NULL.

Pembahasan latihan 1: pesanan terbaru

```
SELECT id, no_document
FROM orders
WHERE deleted_at IS NULL
ORDER BY created_at DESC, id DESC
LIMIT 2;
```

id	no_document
203	ORD-203
202	ORD-202

Filter deleted_at berlaku sebelum pemilihan dua baris teratas.

Pembahasan latihan 2: jumlah pesanan

```
-- 1. Jumlah per status
SELECT order_status_id AS status, COUNT(*) AS total
FROM orders WHERE deleted_at IS NULL
GROUP BY order_status_id ORDER BY order_status_id;

-- 2. Customer dengan > 1 order
SELECT customer_id, COUNT(*) AS total
FROM orders WHERE deleted_at IS NULL
GROUP BY customer_id
HAVING COUNT(*) > 1
ORDER BY customer_id;
```

Hasil data mini:

Soal 1:
Draft = 1
Dikirim = 1
Diterima = 2

Soal 2:
customer_id 3 = 3 order

Untuk label status, JOIN order_statuses. HAVING memfilter jumlah hasil per kelompok.

Pembahasan latihan 2: saldo per fasilitas

```
SELECT entity_id, SUM(qty_on_hand) AS total
FROM stocks
WHERE material_id = 1000 AND activity_id = 1
GROUP BY entity_id
ORDER BY entity_id;
```

entity_id	total
2	800
3	200
4	0

SUM(qty_on_hand) belum tersedia pada tahap WHERE dalam query block ini.

Untuk memilih total > 200:
HAVING SUM(qty_on_hand) > 200

Periksa satuan dan dimensi saldo sebelum agregasi.

Pembahasan akhir 1: fasilitas dengan nol order

```
SELECT e.name, COUNT(o.id) AS total
FROM entities AS e
LEFT JOIN orders AS o
  ON o.customer_id = e.id
 AND o.deleted_at IS NULL
GROUP BY e.id, e.name
ORDER BY e.id;
```

name	total
Dinkes Prov	0
Dinkes Kab	0
PKM Melati	3
PKM Mawar	1

ON mempertahankan seluruh fasilitas. COUNT(o.id) mengabaikan placeholder NULL.

Pembahasan akhir 2: total terkirim minimal 200

```
SELECT e.name, SUM(i.qty_sent) AS sent
FROM entities AS e
JOIN orders AS o ON o.customer_id = e.id
JOIN order_items AS i ON i.order_id = o.id
WHERE o.deleted_at IS NULL AND o.activity_id = 1
  AND i.material_id = 1000
GROUP BY e.id, e.name
HAVING SUM(i.qty_sent) >= 200
ORDER BY sent DESC, e.id;
```

name	sent
PKM Melati	230

WHERE memilih baris yang dihitung. HAVING memilih total customer yang ditampilkan.

Pembahasan akhir 3: material tanpa stok

```
SELECT m.id, m.name FROM materials AS m
WHERE NOT EXISTS (
  SELECT 1 FROM stocks AS s
  WHERE s.material_id = m.id
)
ORDER BY m.id;
```

id	name
1002	Safety box

Hasil ini berarti tidak punya catatan stocks, bukan saldo total sama dengan nol.

Pembahasan akhir 4: laporan dengan CTE

```
WITH sent_summary AS (
  SELECT o.customer_id, SUM(i.qty_sent) AS sent
  FROM orders AS o
  JOIN order_items AS i ON i.order_id = o.id
  WHERE o.deleted_at IS NULL AND o.activity_id = 1
    AND i.material_id = 1000
  GROUP BY o.customer_id
)
SELECT e.name, x.sent
FROM sent_summary AS x
JOIN entities AS e ON e.id = x.customer_id
WHERE x.sent >= 200
ORDER BY x.sent DESC, e.id;
```

name	sent
PKM Melati	230

Query utama dapat memakai WHERE karena sent sudah menjadi kolom hasil CTE.

Bonus: index dan EXPLAIN

```
-- Index ini sudah ada pada DDL bootcamp
-- idx_tx_entity_created (entity_id, created_at)
EXPLAIN
SELECT id, change_qty
FROM transactions
WHERE entity_id = 3
  AND created_at >= TIMESTAMP '2026-02-01'
  AND created_at <  TIMESTAMP '2026-03-01';
```

EXPLAIN menampilkan rencana query.

Index dapat membantu pencarian, tetapi optimizer juga dapat memilih scan tabel.

Data mini terlalu kecil untuk menilai performa produksi.

PostgreSQL tidak otomatis membuat index untuk setiap kolom FK. PK dan UNIQUE memiliki index pendukung.

Bonus: hierarki fasilitas dengan self join

```
SELECT e.name AS child, p.name AS parent
FROM entities AS e
LEFT JOIN entities AS p
  ON p.id = e.parent_entity_id
WHERE e.id IN (3, 4)
ORDER BY e.id;
```

child	parent
PKM Melati	Dinkes Kab
PKM Mawar	Dinkes Kab

Self join membaca satu tingkat induk. Penelusuran banyak tingkat dapat memakai WITH RECURSIVE.

Bonus: risiko UNIQUE dengan batch NULL

```
-- Constraint asli pada stocks
UNIQUE (entity_id, activity_id,
        material_id, batch_id)

-- batch_id tidak memiliki NOT NULL
```

Pada PostgreSQL, UNIQUE biasa menganggap NULL berbeda untuk pemeriksaan keunikan.

Kombinasi entity, activity, material yang sama dengan batch_id NULL bisa muncul berulang.

Desain asli tetap dipakai. Jika NULL harus dianggap satu nilai, evaluasi UNIQUE NULLS NOT DISTINCT.

Bonus: audit saldo dan konsistensi referensi

```
SELECT t.id
FROM transactions AS t
JOIN stocks AS s ON s.id = t.stock_id
WHERE t.entity_id <> s.entity_id
   OR t.material_id <> s.material_id
   OR t.activity_id <> s.activity_id
   OR t.batch_id IS DISTINCT FROM s.batch_id;
```

FK terpisah hanya memastikan setiap ID punya pasangan.

Query ini mencari dimensi mutasi yang berbeda dari dimensi stock_id.

IS DISTINCT FROM membandingkan NULL secara aman.

Audit tidak mengubah data. Tambahkan filter lain bila parent_material_id juga harus konsisten.

Data mini: mutasi material 1000

id	stock_id	opening	change	closing
9001	101	0	180	180
9002	101	180	-60	120
9003	102	0	100	100
9004	102	100	-20	80
9005	103	0	150	150
9006	103	150	-150	0

Contoh riwayat ini terbatas. SUM(change_qty) suatu periode adalah mutasi neto, bukan otomatis saldo stok.

Bonus: stok dengan batch mendekati kedaluwarsa

```
SELECT s.id, b.expired_date
FROM stocks AS s
JOIN batches AS b ON b.id = s.batch_id
WHERE s.material_id = 1000 AND s.activity_id = 1
  AND s.qty_on_hand > 0
  AND b.expired_date >= DATE '2026-06-01'
  AND b.expired_date <  DATE '2026-07-01'
ORDER BY b.expired_date, s.id;
```

id	expired_date
100	2026-06-30
101	2026-06-30

Tanggal acuan latihan: 1 Juni 2026.

Contoh ini memilih batch yang tanggal kedaluwarsanya masih di bulan Juni.

Ambang “mendekati kedaluwarsa” harus ditetapkan oleh aturan laporan. Rentang ini hanya contoh.

Peta singkat penyusunan query

Pertanyaan	Konsep / klausa
Satu baris hasil mewakili apa?	Grain, key, dan satuan
Data berasal dari tabel mana?	FROM dan JOIN
Baris mana yang dihitung?	WHERE dan soft delete
Ringkasan per apa?	GROUP BY dan agregat
Kelompok mana yang tampil?	HAVING
Bagaimana bentuk keluaran?	SELECT, ORDER BY, LIMIT
Perlu beberapa tahap?	Subquery atau CTE

Bedakan saldo, mutasi neto, jumlah terkirim, dan jumlah diterima saat menetapkan metrik.

Referensi dan batas contoh

Struktur tabel dan relasi mengikuti DDL bootcamp yang digunakan dalam kelas.

Target sintaks: PostgreSQL. Cuplikan DDL yang tidak lengkap diberi label.

Semua angka hasil menggunakan data mini fiktif pada slide. Hasil di database kelas mengikuti data generator yang terpasang.

Dokumentasi resmi PostgreSQL per topik tercantum pada speaker notes.

Urutan materi tetap: DDL dan relasi, SELECT dasar, agregasi, JOIN, subquery, lalu CTE.